US Traffic Accidents Analysis

Technical Project Plan

Data Science Capstone

Methodology: CRISP-DM

Tools: PySpark • Pandas • Scikit-Learn • React

Dataset: ~7.7 Million US Accident Records (2016–2023)

Department of Transportation — Road Safety Initiative

1. Project Overview

You have been hired as a data analyst for the Department of Transportation. The DOT is concerned about traffic accident rates across the United States and wants data-driven strategies to reduce accidents and improve road safety.

Your mission is to analyze approximately 7.7 million accident records, identify the most significant patterns and contributing factors, and deliver three actionable insights the DOT can implement. This document outlines the technical approach, step by step.

1.1 What You Will Build

- **Jupyter Notebook:** A single, end-to-end analysis notebook following the CRISP-DM methodology. It starts with PySpark for big-data processing, transitions to Pandas for exploratory analysis and modeling, and concludes with three evidence-backed recommendations.
- **React Dashboard:** A fully interactive web dashboard running on localhost that lets stakeholders explore temporal patterns, geographic hotspots, weather impacts, and model-driven insights through filters, maps, and charts.
- **Executive Summary:** A standalone two-page summary of your three DOT insights with supporting evidence.

1.2 Repository Structure

The project builds on top of the provided capstone template. Here is how the directory expands:

```
DS_CAPSTONE_TEMPLATE/
├── .venv/                          # Python virtual environment
├── Data/
│   ├── US_Accidents.csv              # Raw dataset (~7.7M rows)
│   ├── accidents_sample.parquet      # Stratified sample (~1M rows)
│   └── aggregates/                   # Pre-computed JSON for dashboard
│       ├── temporal_patterns.json
│       ├── geographic_hotspots.json
│       ├── weather_severity.json
│       └── ... (9 files total)
├── Images/                         # Banner and notebook images
├── notebook/
│   └── figures/                      # All saved visualization PNGs
├── dashboard/                      # React application
│   ├── public/data/                  # Copies of aggregate JSONs
│   └── src/components/               # React components
├── outputs/
│   └── executive_summary.md
├── DS_Capstone_Notebook.ipynb       # Main deliverable
├── .gitignore
```

```
├── LICENSE
└── README.md
```

1.3 Technical Architecture

The project uses a two-stage data pipeline. Understanding this architecture is critical before writing any code:

The Two-Stage Pipeline

Stage 1 (PySpark): Processes ALL 7.7 million rows. Handles cleaning, feature engineering, and aggregation at scale. Exports two artifacts: a stratified 1M-row sample (Parquet) and pre-computed summary tables (JSON).

Stage 2 (Pandas): Loads only the 1M-row sample. Performs exploratory analysis, statistical testing, and predictive modeling. This is where your visualizations, hypothesis tests, and classifiers live.

The React dashboard consumes the pre-computed JSON files directly. It never touches raw data, which keeps it fast and eliminates any backend dependency.

Why this split? Because it mirrors how data teams actually work. You use the heavy-duty distributed tool for ETL, then hand off a manageable dataset to the tool with better ergonomics for exploration. Students see both PySpark and Pandas in their natural habitats.

2. Data Pipeline (PySpark)

This phase corresponds to the Data Understanding and Data Preparation sections of your notebook. PySpark processes the full dataset, cleans it, engineers features, then exports what downstream analysis needs.

2.1 Load and Profile the Dataset

Start by loading the CSV and understanding what you have. The profiling step is not just a formality; it directly informs every cleaning decision that follows.

```
# Initialize PySpark with sufficient memory for 7.7M rows
spark = SparkSession.builder \
    .appName('US_Traffic_Accidents') \
    .config('spark.driver.memory', '8g') \
    .getOrCreate()

df = spark.read.csv('./Data/US_Accidents.csv', header=True, inferSchema=True)

# PROFILE: Answer these questions before any cleaning
# 1. How many rows and columns?           -> df.count(), len(df.columns)
# 2. What are the data types?              -> df.printSchema()
# 3. Which columns have nulls, and how many? -> null count per column
# 4. What is the severity distribution?    -> groupBy('Severity').count()
# 5. What date range does the data cover?  -> min/max of Start_Time
# 6. Which states dominate?                -> top 10 by count
```

> **Teaching Moment: Why Profile First?**
>
> Every cleaning decision should be justified by evidence. If you impute Temperature with the state median, you need to know that Temperature has X% nulls and that climate varies by state. The profiling results ARE your justification.

2.2 Data Cleaning

Apply these cleaning operations in sequence. The order matters because later steps depend on earlier ones (for example, timestamp parsing must happen before time-based feature engineering).

Step 1: Drop Columns With Excessive Nulls

Identify columns where more than 60% of values are null. These columns cannot be reliably imputed and would introduce more noise than signal. Typical drops include Number and Wind_Chill(F).

```
# Pseudocode
for each column in dataframe:
    null_pct = count_nulls(column) / total_rows
```

```
    if null_pct > 0.60:
        drop column
        log: 'Dropped {column}: {null_pct}% null'
```

Step 2: Parse Timestamps

Convert Start_Time and End_Time from strings to proper timestamp types. These are the foundation for all temporal analysis and feature engineering.

```
df = df.withColumn('Start_Time', to_timestamp('Start_Time'))
df = df.withColumn('End_Time', to_timestamp('End_Time'))

# Validate: check for nulls introduced by failed parsing
# If nulls increased, inspect the date format and adjust
```

Step 3: Remove Invalid Records

- Rows with null or zero GPS coordinates (Start_Lat, Start_Lng)
- Rows with null Severity (our primary analysis variable)
- Rows with null Start_Time (cannot perform temporal analysis)
- Rows before 2016 (sparse, pre-collection artifacts)

Step 4: Deduplicate

The same accident may appear multiple times if reported by multiple traffic APIs. Deduplicate on the combination of (Start_Lat, Start_Lng, Start_Time, Street). Same location, same time, same street equals same event.

Step 5: Impute Missing Values

The imputation strategy varies by column type and the nature of the missingness:

Column(s)	Strategy	Rationale
Temperature, Humidity, Visibility, Wind Speed, Pressure	State-level median	Climate varies dramatically by region. A global median would assign Arizona temperatures to Alaska accidents.
Wind Direction	Mode per state	Categorical variable. Median is meaningless for cardinal directions.
Weather Condition	Fill with 'Unknown'	Preserve the row for non-weather analysis. Do not fabricate weather data.
Sunrise/Sunset	Derive from hour	Simple heuristic (6am-6pm = Day). Accurate enough for binary analysis.

```
# State-level median imputation (pseudocode)
for col in [Temperature, Humidity, Visibility, WindSpeed, Pressure]:
    state_medians = df.groupBy('State').agg(median(col))
```

```
df = df.join(state_medians, on='State')
df[col] = coalesce(df[col], df[col + '_median'])
df = df.drop(col + '_median')
```

Step 6: Standardize Boolean Columns

The dataset has 14 boolean road-feature columns (Traffic_Signal, Crossing, Junction, etc.). Cast all of them to integer (0/1) for consistent use in statistical tests and modeling.

2.3 Feature Engineering

Create new columns that expose patterns not visible in the raw data. Every engineered feature should have a clear analytical purpose.

Feature	How to Derive	Why It Matters
Hour, DayOfWeek, Month, Year	Extract from Start_Time	Enables temporal pattern analysis (rush hour, seasonal, YoY trends)
IsWeekend	1 if Saturday/Sunday, else 0	Weekend vs. weekday driving behavior differs significantly
TimeOfDay	Bucket Hour into Night (0-5), Morning (6-11), Afternoon (12-17), Evening (18-23)	Coarser time grouping for chi-square tests and visualization
Duration_Min	(End_Time - Start_Time) in minutes. Cap at 1440. Discard negatives.	Measures traffic impact duration. Proxy for disruption severity.
Is_Severe	1 if Severity >= 3, else 0	Binary target variable for logistic regression and random forest
Temp_Bucket, Visibility_Bucket, Wind_Bucket	Categorical bins (e.g., Freezing/Cold/Mild/Warm/Hot)	Groups continuous variables for cross-tabulation and visualization
Road_Feature_Count	Sum of all 14 boolean infrastructure columns	Composite index measuring infrastructure complexity at accident site
Weather_Risk_Score	Count of: low visibility + precipitation + extreme temp + high wind (0–4)	Multi-factor environmental risk index for modeling

2.4 Export: Stratified Sample and Dashboard Aggregates

This is the bridge between the PySpark and Pandas phases. Two exports happen here:

Export 1: Stratified Sample (~1M Rows)

Sample approximately 1 million rows while preserving the severity distribution. This ensures the sample is statistically representative of the full dataset.

```
# Stratified sampling (pseudocode)
target_size = 1_000_000
sampling_fraction = target_size / total_row_count

sample = df.sampleBy(
    col='Severity',
    fractions={1: fraction, 2: fraction, 3: fraction, 4: fraction},
    seed=42
)

# VALIDATE: Compare distributions
# Severity % in sample vs full should match within 1%
# Top-10 state % should match within 2%

sample.toPandas().to_parquet('./Data/accidents_sample.parquet')
```

Export 2: Pre-Computed JSON Aggregates

Compute summary statistics from the FULL dataset (not the sample) and export as JSON. These power the React dashboard. Compute all nine files:

- **temporal_patterns.json** — 7×24 matrix of accident counts (DayOfWeek × Hour) for the heatmap
- **geographic_hotspots.json** — Top 200 cities with lat/lng, count, avg severity, top weather condition
- **weather_severity.json** — Cross-tab of top 15 weather conditions × severity levels
- **infrastructure_factors.json** — Each road feature's accident count, avg severity, % of total
- **severity_distribution.json** — Overall, by year, and by state
- **monthly_trends.json** — Count by year × month with year-over-year growth
- **hourly_patterns.json** — Hourly counts split by weekend/weekday and severity
- **state_summary.json** — Per-state summary table with key statistics
- **model_feature_importance.json** — Placeholder; populated after modeling in Phase 5

Checkpoint: Before Moving to Pandas

✓ accidents_sample.parquet exists and loads correctly (~1M rows)

✓ All 9 JSON files exist and parse without error

✓ Severity distribution in sample matches full dataset within 1-2%

✓ PySpark session can be stopped; Pandas takes over from here

3. Exploratory Data Analysis (Pandas)

This phase loads the 1M-row sample into Pandas and produces 15 visualizations that reveal the story hidden in the data. Every chart should have a descriptive title, labeled axes, and a 2–3 sentence interpretation in the notebook.

3.1 The PySpark-to-Pandas Transition

Add a clear divider in your notebook explaining that PySpark has finished its job. From this point forward, all code uses Pandas on the stratified sample. Once the parquet exists, these cells can be re-run independently without PySpark.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns

plt.style.use('seaborn-v0_8-whitegrid')
SEVERITY_COLORS = {1: '#2ecc71', 2: '#f1c40f', 3: '#e67e22', 4: '#e74c3c'}

df = pd.read_parquet('./Data/accidents_sample.parquet')
```

3.2 Univariate Analysis (Visualizations 1–6)

Start by understanding each variable individually before looking at relationships.

1. **Severity Distribution** — Bar chart with percentage labels, colored by severity level.
2. **Temporal Distributions** — Four-panel figure: (a) accidents by hour with weekday/weekend overlay, (b) by day of week, (c) by month colored by season, (d) by year with trend line.
3. **Top 20 States** — Horizontal bars sorted descending. Annotate with per-capita rate (accidents per 100K population) to reveal which states have disproportionate rates.
4. **Top 20 Cities** — Horizontal bar chart.
5. **Weather Conditions** — Top 15 weather conditions by frequency.
6. **Continuous Distributions** — 2×3 subplot grid with histogram + KDE for temperature, humidity, visibility, wind speed, pressure, duration. Note skewness in each title.

3.3 Bivariate and Multivariate Analysis (Visualizations 7–15)

Now explore how variables interact with each other and with severity:

7. **Hour × DayOfWeek Heatmap** — A 7×24 grid using the YlOrRd colormap. This is one of the most insightful charts in the project. Look for rush-hour peaks, weekend late-night patterns, and early-morning lulls.
8. **Severity vs. Weather** — Grouped bar chart: how does severity distribute across the top 10 weather conditions?

9. **Severity vs. Time of Day** — Stacked bars showing severity proportions across Night/Morning/Afternoon/Evening.
10. **Infrastructure vs. Severity** — For each road feature (Traffic_Signal, Junction, etc.), compute the percentage of accidents that are severe. Horizontal bars, sorted.
11. **Temperature vs. Severity** — Box plots. Annotate with ANOVA result.
12. **Visibility vs. Severity** — Box plots. Annotate with Kruskal-Wallis result.
13. **Geographic Scatter** — Plot 50K random points on lat/lng axes. The point density itself forms the US map shape. Color by severity, alpha=0.05.
14. **Correlation Heatmap** — Numeric columns, upper triangle masked. Highlight |r| > 0.3.
15. **Year-over-Year Monthly Trends** — One line per year (Jan–Dec). Reveals seasonal patterns and growth.

Teaching Moment: Telling a Visual Story

Each visualization should answer a specific question. Do not create a chart just to create a chart. Before coding any plot, write down the question it answers. After generating it, write a 2-3 sentence interpretation. If you cannot interpret the chart, it does not belong in the notebook.

4. Statistical Hypothesis Testing

This phase applies six statistical tests to validate whether the patterns observed in the EDA are statistically significant and practically meaningful. Each test follows the same structure: state hypotheses, run the test, report the statistic, compute the effect size, interpret in plain English.

Critical Concept: Significance vs. Effect Size

With ~1 million rows, almost everything will be statistically significant ($p < 0.05$). This is a mathematical certainty, not an analytical insight. The real question is: how LARGE is the effect?

Always pair p-values with effect sizes. A chi-square test might return $p < 0.001$ but with a Cramer's V of 0.02, which means the association exists but is practically trivial. The DOT cannot act on trivial effects.

Test 1: Severity vs. Time of Day — Chi-Square

Question: Is accident severity independent of time of day, or are certain periods disproportionately dangerous?

```
# Build contingency table: TimeOfDay (rows) x Is_Severe (columns)
contingency = pd.crosstab(df['TimeOfDay'], df['Is_Severe'])

# Run chi-square test of independence
chi2, p, dof, expected = scipy.stats.chi2_contingency(contingency)

# Compute Cramer's V (effect size)
n = contingency.sum().sum()
cramers_v = sqrt(chi2 / (n * (min(contingency.shape) - 1)))
```

Effect size guide: Cramer's V: small = 0.10, medium = 0.30, large = 0.50

Test 2: Temperature across Severity — One-Way ANOVA

Question: Does mean temperature differ significantly across accident severity levels?

```
# Group temperature values by severity level
groups = [group['Temperature(F)'] for _, group in df.groupby('Severity')]

# One-way ANOVA
f_stat, p_value = scipy.stats.f_oneway(*groups)

# Eta-squared effect size
eta_sq = SS_between / SS_total

# If significant, follow up with Tukey HSD for pairwise comparisons
tukey = pairwise_tukeyhsd(df['Temperature(F)'], df['Severity'])
```

Effect size guide: Eta-squared: small = 0.01, medium = 0.06, large = 0.14

Test 3: Visibility across Severity — Kruskal-Wallis

Question: Does visibility differ across severity levels? (Non-parametric, because visibility is right-skewed.)

```
# Kruskal-Wallis: non-parametric alternative to ANOVA
h_stat, p_value = scipy.stats.kruskal(*groups)

# Epsilon-squared effect size
epsilon_sq = (h_stat - k + 1) / (n - k)
```

Test 4: Weekend vs. Weekday Severity — Mann-Whitney U

Question: Do weekend and weekday accidents differ in severity distribution?

```
weekend = df[df['IsWeekend'] == 1]['Severity']
weekday = df[df['IsWeekend'] == 0]['Severity']

u_stat, p_value = mannwhitneyu(weekend, weekday, alternative='two-sided')

# Rank-biserial correlation (effect size for Mann-Whitney)
r = 1 - (2 * u_stat) / (n1 * n2)
```

Test 5: Infrastructure Features — Point-Biserial Correlations

Question: Which road features (traffic signals, crossings, junctions, etc.) are most strongly associated with severe accidents?

```
for feature in boolean_road_feature_columns:
    r, p = pointbiserialr(df[feature], df['Is_Severe'])
    store result: feature, r, p_value, abs(r)

# Sort by absolute correlation descending
# Flag features with |r| > 0.1 as noteworthy
```

Test 6: Seasonal Decomposition

Question: What is the underlying trend and seasonal pattern in monthly accident counts?

```
# Resample to monthly counts, then decompose
```

```
monthly = df.set_index('Start_Time').resample('M').size()

decomposition = seasonal_decompose(monthly, model='additive', period=12)
# Plot: Observed, Trend, Seasonal, Residual components
```

Compile all test results into a summary table at the end of this section. This table becomes a quick reference for the insight synthesis in Phase 6.

5. Predictive Modeling

Build two classifiers to predict whether an accident will be severe (Severity ≥ 3). The goal is not a production model but rather feature importance analysis: which factors matter most for predicting severity? This directly informs the DOT recommendations.

5.1 Feature Preparation

```
# Feature set (21 features total)
TEMPORAL    = ['Hour', 'DayOfWeek', 'Month', 'IsWeekend']
ENVIRONMENT = ['Temperature(F)', 'Humidity(%)', 'Visibility(mi)',
               'Wind_Speed(mph)', 'Pressure(in)']
INFRASTRUCTURE = [all 14 boolean road feature columns]
COMPOSITE   = ['Road_Feature_Count', 'Weather_Risk_Score']

TARGET = 'Is_Severe'

# Train/test split: 80/20, stratified on target, seed=42
# Standard-scale ONLY the continuous features
# Check and report class imbalance ratio
```

Why Two Models?

Logistic Regression provides interpretable coefficients and odds ratios. You can say exactly how much each factor increases the odds of severity. But it assumes linear relationships.

Random Forest captures non-linear interactions and provides permutation-based feature importance. Comparing which features rank high in BOTH models gives robust evidence.

Features that appear in the top 5 of both models are the most defensible recommendations for the DOT.

5.2 Logistic Regression

```
lr = LogisticRegression(
    max_iter=1000,
    class_weight='balanced',  # Handle class imbalance
    random_state=42
)
lr.fit(X_train_scaled, y_train)

# PRODUCE:
# 1. Classification report (precision, recall, F1)
# 2. ROC AUC score
# 3. Confusion matrix heatmap
```

```
# 4. Coefficient bar chart with odds ratios
#    -> odds_ratio = exp(coefficient)
#    -> 'Visibility has OR=0.72, meaning each unit increase
#       reduces odds of severity by 28%'
```

5.3 Random Forest

```
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=15,
    min_samples_leaf=50,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train_unscaled, y_train)  # Trees don't need scaling

# PRODUCE:
# 1. Classification report
# 2. ROC AUC score
# 3. Confusion matrix heatmap
# 4. Impurity-based feature importance (built-in)
# 5. Permutation importance (more reliable):
#    perm = permutation_importance(rf, X_test, y_test, n_repeats=10)
```

5.4 Model Comparison

- **Overlay ROC curves** on a single plot with both AUC values in the legend.
- **Compare feature rankings** side by side. Identify features that rank top-5 in both models.
- **Export feature importance** to model_feature_importance.json for the dashboard.

The modeling section concludes with a narrative paragraph interpreting what the models reveal about controllable vs. uncontrollable risk factors. Infrastructure features are controllable (the DOT can install signals and crossings). Weather is not. This distinction shapes actionable recommendations.

6. Three DOT Insights

This is the payoff. Everything above exists to support three clear, evidence-backed recommendations for the Department of Transportation. Each insight follows the same structure:

Element	What to Include
Finding	1-2 sentence summary of the pattern discovered
Evidence	Statistical test result (test, p-value, effect size) + visualization reference + model ranking
Magnitude	Quantified impact (e.g., '2.3× more likely to be severe', '41% of severe accidents occur during...')
Recommendation	Specific, implementable DOT strategy
Expected Impact	Projected benefit if recommendation is implemented

Suggested Insight Directions

These are starting hypotheses. Let your data confirm or redirect them:

Insight 1: Temporal Risk Windows

Rush hours and low-light periods concentrate the most severe accidents. Supported by the hour-day heatmap, chi-square test on time-of-day, and Hour as a top feature in both models. Recommendation: dynamic speed limits, enhanced lighting, targeted enforcement during identified peak windows.

Insight 2: Environmental Risk Multipliers

Specific combinations of poor weather and low visibility multiply accident severity. Supported by the ANOVA/Kruskal-Wallis tests, weather-severity cross-tabulation, and Weather_Risk_Score importance. Recommendation: automated weather-responsive signage, variable speed advisories, preemptive road treatment.

Insight 3: Infrastructure Feature Gaps

Certain road features (or their absence) disproportionately correlate with severe outcomes. Supported by point-biserial correlations, infrastructure visualization, and feature rankings in the random forest. Recommendation: prioritized audit of high-severity locations, targeted signal and crossing upgrades.

> **Important: Let the Data Lead**
>
> If your analysis reveals a more compelling pattern than these three, go with the data. Perhaps urban vs. rural disparity is striking, or one state's policy produces notably different outcomes. The insights must be earned by the evidence, not forced to fit a template.

Backfill the Overview

Once insights are finalized, go back to the very first section of the notebook (Overview) and write a BLUF (Bottom Line Up Front) that summarizes all three insights in 3–4 sentences. Readers should know your conclusions before they see any code.

7. React Interactive Dashboard

The final deliverable is a fully interactive dashboard that runs locally via npm start. It consumes the pre-computed JSON aggregates, never raw data, so it loads instantly and requires no backend.

7.1 Architecture

```
dashboard/
├── public/data/            # JSON aggregates (copied from Data/aggregates/)
└── src/
    ├── App.jsx               # Root: loads data, manages filter state
    └── components/
        ├── GlobalFilters.jsx  # Year, state, severity, date range
        ├── KPICards.jsx       # 4 summary metric cards
        ├── InteractiveMap.jsx # React-Leaflet map with city markers
        ├── SeverityBreakdown  # Donut chart
        ├── WeatherImpact      # Stacked bar chart
        ├── TemporalHeatmap    # 7x24 Hour-Day grid
        ├── MonthlyTrends      # Multi-year line chart
        ├── FeatureImportance  # Model comparison bars
        └── InsightsPanel      # 3 expandable insight cards
```

7.2 Key Components

Global Filters

All filters apply client-side by filtering the loaded JSON arrays in memory. Year and State are multi-select dropdowns, Severity uses checkboxes, and a date range picker provides month/year granularity. A Reset button clears everything. Filter state lives in the App component and flows down as props.

Interactive Map (React-Leaflet)

Renders the top 200 cities as circle markers on OpenStreetMap tiles. Circle radius scales logarithmically with accident count. Circle color maps to average severity (green through red). Click any marker for a popup with city details.

Temporal Heatmap

A 7-row by 24-column grid built with SVG or D3. Rows represent days of the week, columns represent hours. Color intensity maps from white (few accidents) through yellow, orange, to red (many accidents). Hover any cell for the exact count.

Feature Importance

Horizontal bar chart with a toggle to switch between logistic regression coefficients and random forest permutation importance. Bars are color-coded by feature category (temporal, environmental, infrastructure, composite).

Insights Panel

Three expandable cards at the bottom of the dashboard. Each shows a title and key statistic when collapsed. Clicking expands to reveal the full finding, evidence summary, and recommendation.

7.3 Data Flow

```
// App.jsx pseudocode
on mount:
    load all 9 JSON files in parallel
    store in state

on filter change:
    derive filtered datasets using useMemo
    all child components re-render with filtered data

// No API calls. No database. Just JSON in memory.
```

7.4 Running the Dashboard

```
cd dashboard
npm install
npm start
# Opens at http://localhost:3000
```

Include these instructions and representative screenshots in the notebook's dashboard section so the notebook tells the complete story even without the live application.

8. Execution Summary and Checkpoints

Here is the full execution flow with validation checkpoints. Do not proceed past a checkpoint until it passes.

#	Phase	Key Output	Checkpoint
1	PySpark ETL	Cleaned DataFrame in memory	>5M rows, zero key nulls, severity dist intact
2	Export	Parquet sample + 9 JSON files	~1M rows, distributions match ±2%, JSONs parse
3	EDA	15 visualizations as PNG + inline	All render, none blank, each has interpretation
4	Statistical Tests	6 tests + summary table	All produce valid stats, effect sizes computed
5	Modeling	2 models + importance JSON	AUC > 0.5, JSON exported with status=complete
6	Insights	3 DOT insights + executive summary	Each has stats, magnitude, recommendation
7	Notebook	Complete .ipynb (~69 cells)	Runs top-to-bottom, reads as a report
8	Dashboard	React app on localhost:3000	npm start works, components render, filters function

Key Reminders

- **Correlation is not causation.** This dataset shows conditions at accident time. We have no control group (non-accident conditions), so we cannot compute true risk ratios. Always use language like 'associated with' rather than 'caused by.'
- **Effect size over p-value.** With a million rows, everything is significant. Focus your insights on effects that are large enough to matter for policy decisions.
- **Coverage bias exists.** The data is compiled from traffic APIs. States with better API coverage appear to have more accidents. California's dominance may partly reflect reporting infrastructure, not just road danger.
- **Severity measures disruption, not injury.** The 1-4 scale reflects traffic impact duration. Severity 4 means maximum disruption, not maximum harm to people.
- **The notebook is a narrative.** Every code cell needs context (why are we doing this?) and interpretation (what does this tell us?). A notebook without narrative is a code dump, not an analysis.